

RTC Platform — Product & Technical Overview

Version: 1.0
Date: August 2025
Status: Core platform complete; production hardening next

1. Executive Summary

RTC Platform is an **embeddable real-time communication (RTC) platform** inspired by services like Zego and Agora. It lets any product add **1:1 chat**, **voice calls**, and **group voice** without building WebRTC infrastructure from scratch.

Customers integrate a single JavaScript SDK (`@rtc/sdk`) into their web or mobile apps. The platform handles signaling, authentication, room management, media routing (P2P or SFU), and TURN relay for cross-network connectivity.

What you get today:

- A working monorepo with signaling server, mediasoup SFU, publishable SDK, and live demo
 - Multi-tenant app registry (App ID / App Secret per customer)
 - P2P and SFU voice paths with automatic fallback
 - Redis-backed scaling for signaling
 - PostgreSQL for app credentials
 - coturn TURN server for production-style NAT traversal
-

2. Vision — What We Want to Build

2.1 Problem

Building real-time voice and chat in-house requires deep WebRTC expertise, signaling infrastructure, media servers, TURN relays, auth, scaling, and ongoing maintenance. Most product teams want **“add a voice room in a day”** — not a six-month RTC project.

2.2 Solution

A **hosted RTC platform** with:

Capability	Description
1:1 text chat	Room-based messaging between users in the same session
1:1 voice calls	Direct or SFU-mediated audio between two users
Group voice	Multiple users in a room hear each other via SFU (Selective Forwarding Unit)
Embeddable SDK	Simple API: <code>init</code> , <code>joinRoom</code> , <code>sendMessage</code> , <code>callUser</code> , <code>joinVoiceRoom</code>
Multi-tenant apps	Each customer product gets App ID + Secret (like Zego App Sign)
Secure tokens	Server-side token issuance; secrets never on the client
Scalable signaling	Redis pub/sub for multi-instance WebSocket servers
NAT traversal	STUN + TURN configuration served to clients automatically

2.3 Target Users

- **Product teams** embedding voice/chat into support, telehealth, gaming, or collaboration apps
- **Platform operators** running the RTC backend for multiple customer apps
- **Developers** evaluating RTC options with a runnable demo and clear integration path

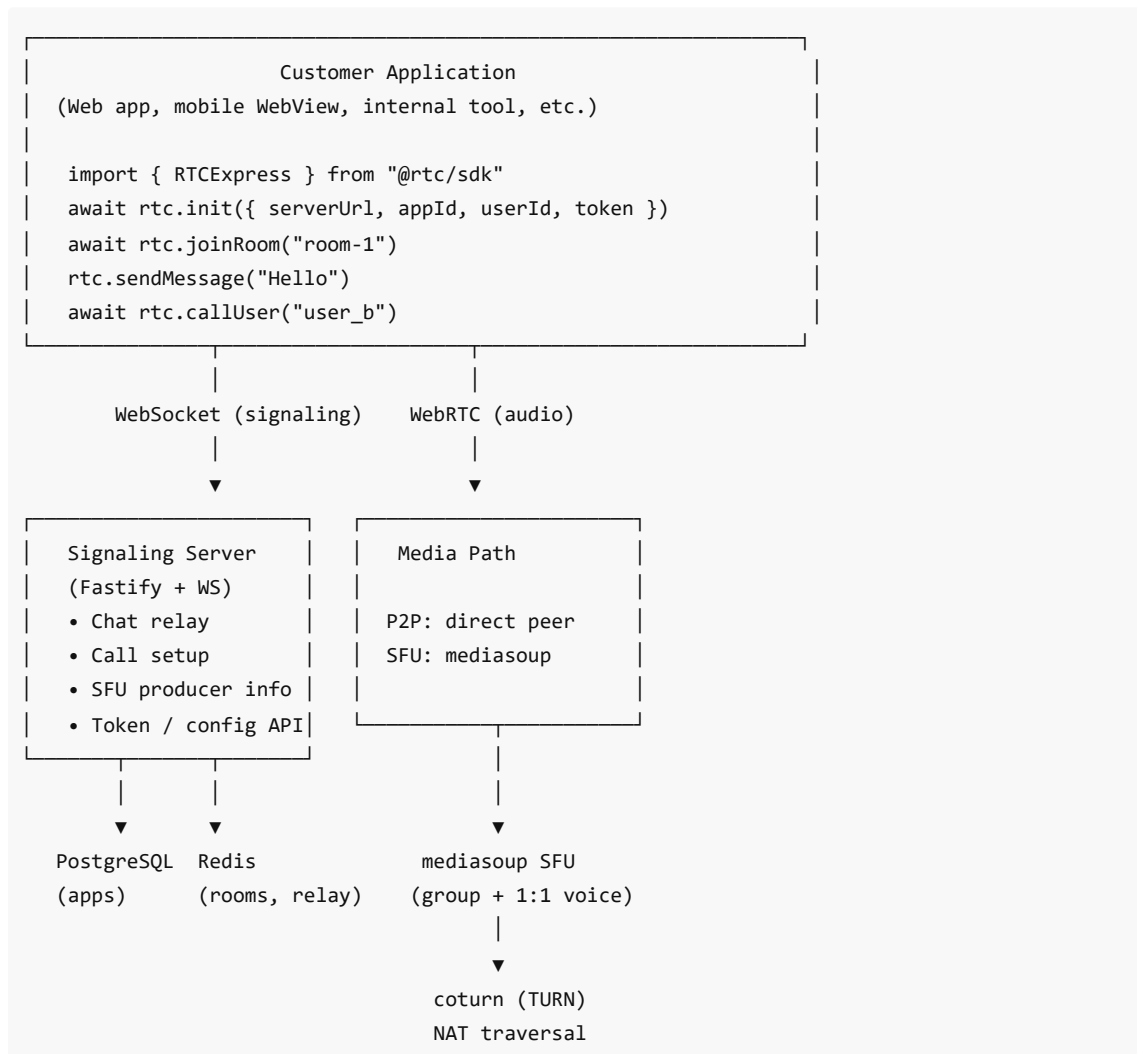
2.4 Business Model (Planned)

Similar to Zego / Agora:

1. Register an app → receive `appId` + `appSecret`
 2. Backend issues short-lived JWT tokens for users
 3. SDK connects to signaling + media services
 4. Usage metering (minutes, concurrent users) — **future phase**
-

3. How It Works — End-to-End Flow

3.1 High-Level Architecture



3.2 User Journey — Chat + Voice

Step 1 — Backend authentication (server-side only)

Your backend calls the signaling API with `appId` , `appSecret` , and `userId` . It receives a JWT token (typically 1 hour TTL). Never expose `appSecret` in client code.

Step 2 — SDK initialization

The client SDK loads platform config (`GET /v1/config`): ICE/TURN servers, SFU URL, feature flags. It opens a WebSocket to `/ws?token=...` .

Step 3 — Join a room

Both users join the same `roomId` . Signaling tracks presence in Redis (or in-memory for dev). Other members receive `user_joined` events.

Step 4 — Text chat

`sendMessage` broadcasts to all room members via signaling. No media server involved.

Step 5 — 1:1 voice call

Caller invokes `callUser(peerId)` . Callee receives `callInvite` . On accept:

- **P2P mode:** WebRTC offer/answer + ICE candidates relayed through signaling
- **SFU mode:** Each side publishes audio to mediasoup; remote audio consumed via SFU

Step 6 — Group voice

`joinVoiceRoom()` connects all room members through the SFU. Everyone’s microphone is mixed at the client; server forwards individual streams (SFU pattern). `leaveVoiceRoom()` disconnects media only; chat room membership remains.

3.3 Media Modes

Mode	When used	Behavior
auto	Default	Use SFU if configured and available; otherwise P2P
sfu	Group calls, controlled routing	Always mediasoup; falls back to P2P if SFU offline
p2p	Low-latency 1:1, dev/testing	Direct WebRTC between peers

4. Platform Components

4.1 Monorepo Structure

```
rtc-platform/
├─ packages/
│  ├─ protocol/      Shared TypeScript message types (client ↔ server)
│  ├─ signaling/     WebSocket + REST API + app registry
│  ├─ sdk/           @rtc/sdk – customer-facing embeddable library
│  ├─ media-sfu/     mediasoup SFU service
│  └─ demo/          Two-panel integration demo (Vite)
├─ docker-compose.yml PostgreSQL, Redis, coturn
└─ docs/            Product documentation
```

4.2 Signaling Server (@rtc/signaling)

- **Runtime:** Node.js, Fastify, WebSocket (ws)
- **Port:** 4000 (default)
- **Responsibilities:**
 - JWT token issuance (POST /v1/token)
 - Platform config (GET /v1/config) — ICE, SFU URL, features
 - WebSocket signaling — rooms, messages, call state, WebRTC relay
 - Admin API — register/list customer apps
- **Persistence:**
 - PostgreSQL — app registry (app_id , hashed secret)
 - Redis — room membership, cross-instance message relay
- **Security:** bcrypt-hashed app secrets, rate limiting, admin API key

4.3 Media SFU (@rtc/media-sfu)

- **Runtime:** Node.js + mediasoup
- **Port:** 4100 (default)
- **Responsibilities:**
 - Per-room mediasoup routers
 - WebRTC transports, audio producers/consumers
 - REST API for join room, create transport, produce, consume
- **Use cases:** 1:1 voice (SFU mode) and group voice (N participants)

4.4 SDK (@rtc/sdk)

- **Package name:** @rtc/sdk
- **Build output:** ESM + CJS + TypeScript types (packages/sdk/dist)
- **Main class:** RTCExpress
- **Key methods:**
 - init(options) — connect and load config
 - joinRoom(roomId) / leaveRoom()
 - sendMessage(text)
 - callUser(userId) / acceptCall() / rejectCall() / endCall()
 - muteMicrophone(muted)
 - joinVoiceRoom() / leaveVoiceRoom()
 - getMediaMode() — returns "p2p" or "sfu"
- **Events:** connected , message , callInvite , callState , voiceRoomJoined , userJoined , userLeft , error

4.5 Demo Application

- **Port:** 5180
- **Purpose:** Side-by-side User A / User B panels
- **Features:** Connect, join room, chat, 1:1 call, group voice, media mode selector
- **Use:** QA, sales demos, developer onboarding

4.6 Infrastructure (Docker Compose)

Service	Purpose
PostgreSQL 16	App registry database

Redis 7	Signaling state + pub/sub
coturn	TURN relay for WebRTC across firewalls

5. API Reference (Summary)

Public APIs

Endpoint	Method	Description
/v1/token	POST	Issue user JWT (appId, appSecret, userId)
/v1/config	GET	ICE servers, SFU URL, feature flags
/ws?token=	WebSocket	Real-time signaling

Admin APIs (require `x-admin-key`)

Endpoint	Method	Description
/v1/admin/apps	POST	Create new customer app
/v1/admin/apps	GET	List registered apps

SFU APIs

Endpoint	Method	Description
/v1/rooms/:id/join	POST	Join mediasoup room
Transport / produce / consume	POST	WebRTC media negotiation

WebSocket Message Types (Protocol)

Client → Server: `join_room` , `leave_room` , `send_message` , `call_invite` , `call_accept` , `call_reject` , `call_end` , `webrtc_offer` , `webrtc_answer` , `ice_candidate` , `sfu_producer`

Server → Client: `connected` , `room_joined` , `user_joined` , `user_left` , `message` , `call_invite` , `call_state` , `sfu_producer` , `error`

6. Security Model

1. **App Secret** — Stored hashed in PostgreSQL; used only server-side for token requests
2. **User Token** — Short-lived JWT scoped to `appId` + `userId` (+ optional `roomId`)
3. **WebSocket** — Requires valid token; all actions tied to authenticated user
4. **Admin API** — Separate `ADMIN_API_KEY` for app registration
5. **TURN** — Time-limited credentials can be added in production (current dev uses static creds)

7. Demo Credentials (Development)

Key	Value
-----	-------

App ID	demo-app
App Secret	demo-secret
Admin API Key	dev-admin-key
TURN user	rtc / rtc-turn-secret

8. Running the Platform Locally

Prerequisites

- Node.js 18+
- Docker (for PostgreSQL, Redis, coturn)
- npm

Quick Start

```
cd rtc-platform
npm install
npm run dev:infra      # Start postgres, redis, coturn
cp packages/signaling/.env.example packages/signaling/.env
npm run dev            # Signaling + SFU + demo
```

Service URLs

Service	URL
Signaling	http://localhost:4000
SFU	http://localhost:4100
Demo	http://localhost:5180

Demo Walkthrough

1. Open <http://localhost:5180> in two browser tabs (or windows)
2. Set User A = `user_a` , User B = `user_b` , same room `room-1`
3. Click **Connect** then **Join room** on both
4. Send chat messages — they appear on both sides
5. Enter peer ID and click **1:1 Call** — accept on the other side
6. Click **Join group voice** on both — all room members hear each other

Build SDK for Distribution

```
npm run build:sdk
# Output: packages/sdk/dist/
```

9. Integration Example

```
import { RTCEExpress } from "@rtc/sdk";

// Server-side: fetch token (never expose appSecret to client)
const { token } = await RTCEExpress.fetchToken("https://signal.example.com", {
  appId: "your-app-id",
  appSecret: process.env.RTC_APP_SECRET,
  userId: "user_123",
});

// Client-side
const rtc = new RTCEExpress();
await rtc.init({
  serverUrl: "https://signal.example.com",
  appId: "your-app-id",
  userId: "user_123",
  token,
  mediaMode: "auto",
});

rtc.on("message", ({ fromUserId, text }) => {
  console.log(`${fromUserId}: ${text}`);
});

rtc.on("callInvite", ({ fromUserId }) => {
  showIncomingCallUI(fromUserId);
});

await rtc.joinRoom("support-42");
rtc.sendMessage("Hello, how can I help?");

await rtc.callUser("agent_7");
await rtc.acceptCall();
rtc.muteMicrophone(false);
await rtc.endCall();

await rtc.joinVoiceRoom(); // Group voice
rtc.leaveVoiceRoom();
```

10. Development Roadmap

Completed 

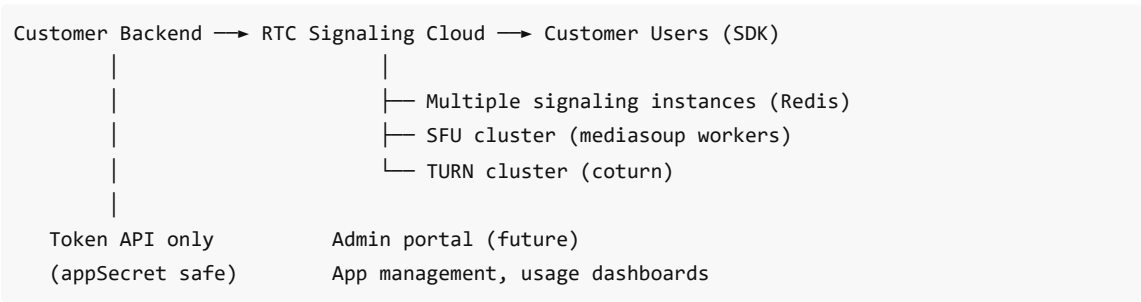
Phase	Deliverable
Phase 1	P2P chat + 1:1 voice via WebRTC
Phase 2	Redis multi-node signaling relay
Phase 3	TURN + ICE config API, PostgreSQL app registry, admin API
Phase 4	mediasoup SFU service, SDK SFU voice path, group voice

Next Steps SOON

Item	Description
npm publish	Publish @rtc/sdk to npm registry
Production hardening	TLS everywhere, secret rotation, health checks
Video	Camera tracks through SFU
Mobile SDKs	React Native / Flutter wrappers
Usage analytics	Minutes, peak concurrency, per-app billing
Kubernetes deploy	Helm charts for signaling + SFU + autoscaling
Token-scoped TURN	Dynamic TURN credentials per session

11. Expected Final Product

When fully deployed for production customers, the platform will operate as:



Customer experience:

1. Sign up → receive App ID and Secret
2. Add @rtc/sdk to their frontend
3. Add one backend endpoint to mint tokens
4. Users get chat + voice in minutes

Operator experience:

1. Run docker-compose or K8s stack
2. Register customer apps via admin API
3. Monitor Redis, SFU workers, and TURN capacity
4. Scale signaling horizontally; SFU per region as needed

12. Technical Requirements for Production

Requirement	Notes
HTTPS	Required for getUserMedia (except localhost)
Public IP / TURN	Required for users behind strict NAT

UDP ports	SFU uses 40000–40100 (configurable)
PostgreSQL	Required for multi-tenant app registry
Redis	Required for multi-instance signaling
JWT secret	Must be strong and rotated in production

13. Contact & Repository

- **Repository:** `rtc-platform` (local monorepo)
 - **Primary packages:** `@rtc/sdk` , `@rtc/signaling` , `@rtc/media-sfu` , `@rtc/protocol`
 - **Documentation:** `README.md` , `packages/sdk/README.md` , this document
-

This document describes the RTC Platform as built and the path to a production-ready embeddable RTC service comparable to Zego-style offerings.